

LAPORAN PRAKTIKUM 3
PEMOGRAMAN BERORIENTASI OBJEK
KEAMANAN & INTEGRITAS DATA (ENKAPSULASI)



Disusun Oleh:
Annisa Layli Ramadhani
2511532024

Dosen Pengampu:
Nurfiah, S.St, M.Kom

Asisten Praktikum:
Muhammad Dawi Syauqi

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

A. Langkah-Langkah

1. Mengunci Entitas Transaksi

a. Buat Class Transaksi

```
public class Transaksi_2511532024 {
```

Membuat *class* bernama *Transaksi_2511532024*. *Class* ini digunakan untuk menyimpan dan mengelola data transaksi pada sistem perbankan.

```
private String idTransaksi;
```

Membuat atribut *idTransaksi* dengan tipe data *String* untuk menyimpan *ID* atau kode transaksi. *private* digunakan agar atribut tidak dapat diakses atau diubah secara langsung dari luar *class*.

```
private String jenis;
```

Membuat atribut *jenis* dengan tipe data *String* yang digunakan untuk menyimpan jenis transaksi, seperti *Kredit* untuk setor tunai dan *Debit* untuk tarik tunai.

```
private double nominal;
```

Membuat atribut *nominal* dengan tipe data *double* untuk menyimpan jumlah uang dalam suatu transaksi. Atribut dibuat *private* agar data tidak dapat dimanipulasi secara langsung dari luar *class*.

b. Constructor

```
public Transaksi_2511532024(String id, String jenis, double nominal) {
```

Membuat *constructor* *Transaksi_2511532024* yang digunakan untuk memberikan nilai awal pada objek transaksi ketika objek dibuat. *Constructor* menerima tiga parameter, yaitu *id*, *jenis*, dan *nominal*.

<i>this.idTransaksi = id;</i>

Memasukkan nilai parameter id ke dalam atribut <i>idTransaksi</i> . Kata <i>this</i> menunjukkan bahwa <i>idTransaksi</i> merupakan atribut milik objek yang sedang dibuat.

<i>this.jenis = jenis;</i>

Memasukkan nilai parameter jenis ke dalam atribut <i>jenis</i> pada objek transaksi.
--

<i>this.nominal = nominal;</i>

Memasukkan nilai parameter nominal ke dalam atribut <i>nominal</i> pada objek transaksi.
--

c. Getter ID Transaksi

<i>public String getIdTransaksi() {</i>

Membuat <i>method</i> <i>getIdTransaksi()</i> yang digunakan untuk mengambil atau membaca nilai atribut <i>idTransaksi</i> . <i>Method</i> dibuat <i>public</i> agar dapat dipanggil dari <i>class</i> lain.
--

<i>return idTransaksi;</i>

Mengembalikan nilai yang tersimpan pada atribut <i>idTransaksi</i> .
--

d. Getter Jenis Transaksi

<i>public String getJenis() {</i>

Membuat <i>method</i> <i>getJenis()</i> yang digunakan untuk mengambil nilai dari atribut <i>jenis</i> .
--

<i>return jenis;</i>

Mengembalikan nilai yang tersimpan pada atribut <i>jenis</i> .
--

e. Getter Nominal

```
public double getNominal() {
```

Membuat method *getNominal()* yang digunakan untuk mengambil nilai dari atribut nominal.

```
return nominal;
```

Mengembalikan nilai nominal transaksi yang tersimpan pada objek.

f. Method Cetak Detail

```
public void cetakDetail() {
```

Membuat method *cetakDetail()* yang digunakan untuk menampilkan informasi lengkap dari suatu transaksi. Tipe *void* menunjukkan bahwa *method* ini tidak mengembalikan nilai.

```
System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " |  
Nominal: Rp" + nominal);
```

Menampilkan ID transaksi, jenis transaksi, dan nominal transaksi ke *console*. Tanda + digunakan untuk menggabungkan teks dengan nilai yang terdapat pada masing-masing atribut.

2. Mengunci Entitas Rekening dan Menambahkan PIN

a. Mengunci Atribut dengan Private

```
private String nomorRekening;
```

Membuat atribut *nomorRekening* dengan tipe data *String* untuk menyimpan nomor rekening. Atribut dibuat *private* agar tidak dapat diakses atau diubah secara langsung dari luar *class*.

```
private String namaPemilik;
```

Membuat atribut *namaPemilik* dengan tipe data *String* untuk menyimpan nama pemilik rekening.

```
private double saldo;
```

Membuat atribut *saldo* dengan tipe data *double* untuk menyimpan jumlah saldo yang dimiliki oleh rekening.

```
private String pin;
```

Membuat atribut *pin* dengan tipe data *String* untuk menyimpan PIN rekening. Atribut dibuat *private* agar PIN tidak dapat diakses secara langsung dari luar *class*.

```
ArrayList<Transaksi_2511532024> riwayatTransaksi;
```

Membuat *ArrayList* bernama *riwayatTransaksi* untuk menyimpan banyak objek *Transaksi_2511532024*. *ArrayList* ini digunakan untuk menyimpan seluruh riwayat transaksi yang dilakukan oleh suatu rekening.

b. Modifikasi Constructor untuk Menerima PIN Awal

```
public Rekening_2511532024(String nomor, String nama, double saldoAwal, String pinAwal) {
```

Membuat *constructor Rekening_2511532024* yang digunakan untuk memberikan nilai awal ketika objek rekening dibuat. *Constructor* menerima empat parameter, yaitu nomor rekening, nama pemilik, saldo awal, dan PIN awal.

```
this.nomorRekening = nomor;
```

Memasukkan nilai parameter nomor ke dalam atribut *nomorRekening*.

```
this.namaPemilik = nama;
```

Memasukkan nilai parameter nama ke dalam atribut *namaPemilik*.

```
this.saldo = saldoAwal;
```

Memasukkan nilai parameter <i>saldoAwal</i> ke dalam atribut saldo.

c. Validasi PIN di dalam Constructor

<i>if (pinAwal.length() == 6) {</i>

Memeriksa panjang PIN menggunakan <i>length()</i> . Jika jumlah karakter pada <i>pinAwal</i> sama dengan 6, maka PIN dianggap memenuhi ketentuan.

<i>this.pin = pinAwal;</i>

Jika PIN memiliki panjang 6 karakter, nilai <i>pinAwal</i> disimpan ke dalam atribut <i>pin</i> .

<i>} else {</i>

Bagian <i>else</i> dijalankan jika panjang PIN tidak sama dengan 6 karakter.
--

<i>System.out.println("Peringatan: PIN harus 6 digit! menggunakan PIN default 123456");</i>

Menampilkan pesan peringatan bahwa PIN harus terdiri dari 6 digit. Jika PIN yang dimasukkan tidak memenuhi ketentuan, program akan menggunakan PIN default 123456.
--

<i>this.pin = "123456";</i>

Mengatur nilai atribut <i>pin</i> menjadi "123456" sebagai PIN <i>default</i> ketika PIN awal tidak memiliki 6 karakter.
--

<i>this.riwayatTransaksi = new ArrayList<>();</i>

Membuat objek <i>ArrayList</i> baru untuk menyimpan riwayat transaksi rekening. Setiap rekening memiliki tempat penyimpanan riwayat transaksinya sendiri.

<code>System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");</code>

Menampilkan pesan bahwa rekening berhasil dibuat dan menampilkan nama pemilik rekening. Tanda + digunakan untuk menggabungkan teks dengan nilai <i>namaPemilik</i> .
--

d. Getter untuk Atribut yang diizinkan dibaca publik

<code>public String getNomorRekening() {</code>

Membuat <i>method</i> <i>getNomorRekening()</i> yang digunakan untuk mengambil atau membaca nilai atribut <i>nomorRekening</i> .
--

<code>return nomorRekening;</code>

Mengembalikan nilai nomor rekening yang tersimpan pada atribut <i>nomorRekening</i> .

<code>public String getNamaPemilik() {</code>

Membuat <i>method</i> <i>getNamaPemilik()</i> yang digunakan untuk mengambil atau membaca nama pemilik rekening.
--

<code>return namaPemilik;</code>

Mengembalikan nilai yang tersimpan pada atribut <i>namaPemilik</i> .
--

e. Method Otentifikasi Internal (Validasi Enkapsulasi)

<code>public boolean otentikasi(String inputPin) {</code>

Membuat <i>method</i> <i>otentikasi()</i> yang digunakan untuk memeriksa apakah PIN yang dimasukkan pengguna sesuai dengan PIN rekening. <i>Method</i> mengembalikan nilai <i>boolean</i> , yaitu true jika PIN benar dan false jika PIN salah.

<code>return this.pin.equals(inputPin);</code>
--

Membandingkan PIN yang tersimpan pada atribut <i>pin</i> dengan PIN

yang dimasukkan pengguna melalui *inputPin*. *Method equals()* digunakan untuk membandingkan isi dari dua *String*. Jika kedua PIN sama, hasilnya *true*, sedangkan jika berbeda hasilnya *false*.

B. Tugas/Latihan Praktikum

1. Menu 1 (Buka Rekening) : Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).

a. Class Main.

```
case 1:
System.out.print("Masukkan No Rekening: ");
String no = input.nextLine();

System.out.print("Masukkan Nama Pemilik: ");
String nama = input.nextLine();

System.out.print("Masukkan Saldo Awal: ");
double saldo = input.nextDouble();
input.nextLine();

System.out.print("Masukkan PIN (6 Digit): ");
String pin = input.nextLine();

if (pin.length() != 6) {
System.out.println("PIN harus terdiri dari 6 Digit!");
break;
}

Rekening_2511532024 rekeningBaru = new
Rekening_2511532024(no, nama, saldo, pin);
daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
System.out.println("Rekening berhasil ditambahkan.");
break;
```

Penjelasan Code:

a. Class Main

```
System.out.print("Masukkan PIN (6 Digit): ");
```

Menampilkan perintah kepada pengguna untuk memasukkan PIN yang terdiri dari 6 digit.

```
String pin = input.nextLine();
```

Membaca PIN yang dimasukkan pengguna menggunakan *nextLine()* dan menyimpannya ke dalam variabel *pin* dengan tipe

<code>data String.</code>

<code>if (pin.length() != 6) {</code>

Memeriksa jumlah karakter PIN menggunakan <code>length()</code> . Jika panjang PIN tidak sama dengan 6 karakter, maka kondisi <code>if</code> dijalankan.

<code>System.out.println("PIN harus terdiri dari 6 Digit!");</code>

Menampilkan pesan bahwa PIN yang dimasukkan harus terdiri dari 6 digit.

<code>break;</code>

Menghentikan proses pada <i>case</i> yang sedang dijalankan. Program kemudian kembali ke perulangan menu utama sehingga rekening tidak jadi dibuat.

<code>Rekening_2511532024 rekeningBaru = new Rekening_2511532024(no, nama, saldo, pin);</code>
--

Membuat objek rekening baru bernama <i>rekeningBaru</i> dari <i>class Rekening_2511532024</i> . Data yang diberikan adalah nomor rekening no, nama pemilik nama, saldo awal saldo, dan PIN pin.

<code>daftarRekening.add(rekeningBaru);</code>
--

Menambahkan objek <i>rekeningBaru</i> ke dalam <i>ArrayList daftarRekening</i> , sehingga rekening tersebut tersimpan dalam daftar rekening.
--

<code>akunAktif = rekeningBaru;</code>
--

Menjadikan rekening yang baru dibuat sebagai <i>akunAktif</i> , yaitu rekening yang sedang digunakan oleh pengguna.

```
System.out.println("Rekening berhasil ditambahkan.");
```

Menampilkan pesan bahwa rekening berhasil ditambahkan ke dalam sistem.

```
break;
```

Menghentikan proses *case* setelah rekening berhasil dibuat dan ditambahkan. Program kemudian kembali ke menu utama.

Output

```
=== SISTEM PERBANKAN MINI ===  
  
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cek Mutasi (Riwayat)  
0. Keluar  
Pilih Menu: 1  
Masukkan No Rekening: 001  
Masukkan Nama Pemilik: nisa  
Masukkan Saldo Awal: 10000000  
Masukkan PIN (6 Digit): 123456  
Rekening atas nama nisa berhasil dibuat.  
Rekening berhasil ditambahkan.
```

2. Keamanan Transaksi

Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.

- Panggil fungsi `akunAktif.otentikasi(pinYangDiinput)`.
- Jika mengembalikan nilai `true`, jalankan transaksi.
- Jika `false`, batalkan dan cetak: "Akses Ditolak: PIN yang anda masukkan salah!".

a. Case 3

```
case 3:  
if (akunAktif == null) {  
System.out.println("Error: Mohon maaf, Anda belum  
memiliki nomor rekening!");  
} else {  
System.out.print("Masukkan PIN: ");
```

```
String pinTarik = input.nextLine();
if (akunAktif.otentikasi(pinTarik)) {
System.out.print("Masukkan nominal tarik tunai: ");
double tarik = input.nextDouble();
input.nextLine();

akunAktif.tarikTunai(tarik); // Memanggil Behavior /
method
} else {
System.out.println("Akses ditolak: PIN yang anda
masukkan salah!");
}
}
break;
```

b. Case 6

```
case 6:
System.out.print("Masukkan PIN: ");
String pinMutasi = input.nextLine();
if (akunAktif.otentikasi(pinMutasi)) {
akunAktif.cetakMutasi();
} else {
System.out.println("Akses ditolak PIN yang anda
masukkan salah!");
}
break;
```

Penjelasan Code:

a. Case 3

<code>System.out.print("Masukkan PIN: ");</code>
--

Meminta pengguna memasukkan PIN sebelum melakukan transaksi tarik tunai.
--

<code>String pinTarik = input.nextLine();</code>
--

Membaca PIN yang dimasukkan pengguna dan menyimpannya ke dalam variabel <i>pinTarik</i> .

<code>if (akunAktif.otentikasi(pinTarik)) {</code>
--

Memanggil <i>method otentikasi()</i> pada rekening aktif untuk memeriksa apakah PIN yang dimasukkan sesuai dengan PIN rekening. Jika hasilnya <i>true</i> , pengguna dapat melanjutkan transaksi.

```
System.out.print("Masukkan nominal tarik tunai: ");
```

Meminta pengguna memasukkan jumlah uang yang ingin ditarik.

```
double tarik = input.nextDouble();
```

Membaca nominal tarik tunai yang dimasukkan pengguna dan menyimpannya ke dalam variabel tarik dengan tipe data *double*.

```
input.nextLine();
```

Membersihkan sisa karakter Enter setelah penggunaan *nextDouble()*, sehingga input berikutnya dapat dibaca dengan benar.

```
akunAktif.tarikTunai(tarik); // Memanggil Behavior / method
```

Memanggil *method tarikTunai()* dari objek *akunAktif* dengan nilai tarik sebagai parameter. *Method* ini digunakan untuk menjalankan proses penarikan tunai.

```
} else {
```

Bagian ini dijalankan jika hasil *otentikasi()* adalah *false*, yang berarti PIN yang dimasukkan tidak sesuai.

```
System.out.println("Akses ditolak: PIN yang anda masukkan salah!");
```

Menampilkan pesan bahwa akses transaksi ditolak karena PIN yang dimasukkan salah.

```
break;
```

Menghentikan proses *case 3* dan kembali ke menu utama.

b. Case 6.

```
System.out.print("Masukkan PIN: ");
```

Meminta pengguna memasukkan PIN sebelum dapat melihat riwayat atau mutasi transaksi rekening.

```
String pinMutasi = input.nextLine();
```

Membaca PIN yang dimasukkan pengguna dan menyimpannya ke dalam variabel *pinMutasi*.

```
if (akunAktif.otentikasi(pinMutasi)) {
```

Memanggil *method otentikasi()* untuk memeriksa apakah PIN yang dimasukkan sesuai dengan PIN rekening aktif. Jika benar, program akan melanjutkan ke proses mencetak mutasi.

```
akunAktif.cetakMutasi();
```

Memanggil *method cetakMutasi()* pada rekening aktif untuk menampilkan riwayat transaksi yang tersimpan dalam rekening tersebut.

```
} else {
```

Bagian *else* dijalankan apabila PIN yang dimasukkan tidak sesuai dengan PIN rekening.

```
System.out.println("Akses ditolak PIN yang anda masukkan salah!");
```

Menampilkan pesan bahwa akses untuk melihat mutasi ditolak karena PIN yang dimasukkan salah.

```
break;
```

Menghentikan proses *case 6* dan mengembalikan program ke menu utama.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
0. Keluar
Pilih Menu: 3
Masukkan PIN: 123456
Masukkan nominal tarik tunai: 100000
Tarik tunai Rp100000.0 berhasil. Saldo saat ini: Rp900000.0
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
0. Keluar
Pilih Menu: 3
Masukkan PIN: 111111
Akses ditolak: PIN yang anda masukkan salah!
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
0. Keluar
Pilih Menu: 6
Masukkan PIN: 123456
--- MUTASI REKENING ---
ID: TRX-T-1790514328422 | Jenis: Debit | Nominal: Rp100000.0
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
0. Keluar
Pilih Menu: 6
Masukkan PIN: 111111
Akses ditolak PIN yang anda masukkan salah!
```

C. Link Repository GitHub

https://github.com/annisalayli109/PrakPBO_B_2511532024/tree/75990549ffed7be444e85a3c6835602c4f05312b/PrakPBO_B_2511532024/src/Pekan3_2511532024